

Industrial Internship Report on

“AutoParts Hub — Automotive Parts Management System”

Prepared by

Mumo

UCT Full Stack Internship Projects — UpSkill Campus

Executive Summary

This report provides details of the Industrial Internship offered by UpSkill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT). The internship is centred on a project/problem statement provided by UCT, to be completed — including this report — within the internship period.

My project is AutoParts Hub, a full-stack web application for managing automotive parts. It gives automotive-parts businesses a centralised, user-friendly system for organising products, categories, vehicle information and stock records, replacing disconnected manual records with a searchable, maintainable application built on a React frontend, a Flask REST API backend, and a MySQL database.

[This section is a working draft — to be updated with a closing summary of outcomes and overall reflection once the remaining weeks of implementation and testing are complete.]

Preface

[Placeholder — to be completed once all internship weeks are finished. Suggested content:]

- A summary of the work completed across all weeks of the internship.
- Why an internship like this matters for career development — the shift from academic exercises to a real project brief, timeline and stakeholders.
- A brief recap of the AutoParts Hub problem statement and what the project sets out to solve.
- The opportunity provided by UpSkill Campus and UCT, and how the programme was structured week by week.
- Overall learnings and experience across the internship.
- Acknowledgements — supervisors, mentors and peers who provided support (to be named).
- A closing message to junior learners following the same programme.

Introduction

About UniConverge Technologies Pvt Ltd

UniConverge Technologies Pvt Ltd (UCT) is a company established in 2013, working in the Digital Transformation domain and providing industrial solutions with a primary focus on sustainability and return on investment (RoI). To develop its products and solutions, UCT leverages cutting-edge technologies including the Internet of Things (IoT), cybersecurity, cloud computing (AWS, Azure), machine learning, communication technologies (4G/5G/LoRaWAN), Java Full Stack, Python and front-end development.

UCT's product portfolio includes the UCT Insight IoT platform (built with a Java backend and a ReactJS frontend, supporting MySQL and various NoSQL databases, with device connectivity via MQTT, CoAP, HTTP, Modbus TCP and OPC UA), the Factory Watch smart factory platform for production and asset monitoring, LoRaWAN-based solutions for agritech, smart cities and utility metering, and predictive maintenance solutions combining embedded systems, industrial IoT and machine learning.

About UpSkill Campus (USC)

UpSkill Campus, along with The IoT Academy and in association with UniConverge Technologies, facilitates the smooth execution of the complete internship process. USC is a career development platform delivering personalised, self-paced upskilling alongside additional support services such as internships, projects, interaction with industry experts and career growth services, with the broader aim of upskilling 1 million learners over the next 5 years.

The IoT Academy

The IoT Academy is the EdTech division of UCT, running long-form executive certification programmes in collaboration with EICT Academy, IITK, IITR and IITG across multiple domains.

Objectives of this Internship Programme

- Gain practical experience of working in the industry.
- Solve real-world problems.
- Improve job prospects.
- Deepen understanding of the field and its applications.
- Support personal growth — better communication and problem-solving.

Reference

[1] — to be added

[2] — to be added

[3] — to be added

Glossary

Term	Acronym / Meaning
AutoParts Hub	Project name for the automotive parts management system
ORM	Object-Relational Mapping (used via Flask-SQLAlchemy)
API	Application Programming Interface
SKU	Stock Keeping Unit

Problem Statement

Automotive-parts businesses handle products, categories, vehicle information and stock records that need to be organised and accessed efficiently. Many such businesses currently rely on disconnected records — spreadsheets, paper logs or siloed systems — that make it difficult to search inventory, track stock levels accurately, or associate parts with the correct vehicles.

The AutoParts Hub project addresses this by providing a centralised full-stack web application through which categories, products and vehicle information can be managed through a single, user-friendly interface, rather than relying on disconnected records.

Existing and Proposed Solution

Many automotive-parts businesses currently rely on spreadsheets, paper logs or generic, non-specialised inventory tools. These typically offer no structured link between parts, categories and the vehicles they fit, provide weak search across brand/SKU/description, and give no at-a-glance visibility into stock health or inventory value — all of which slow down day-to-day operations and make it easy for stock discrepancies to go unnoticed.

AutoParts Hub's proposed — and now partly implemented — solution is a centralised full-stack web application with:

- A relational data model connecting Products, Categories and Vehicles (MySQL via SQLAlchemy ORM).
- A dashboard giving at-a-glance KPIs: total products, total inventory value, total units in stock, and a live low-stock alert count (products at or below a 10-unit threshold).
- A searchable, filterable product catalogue (by name, brand, SKU or description) with category browsing.
- An Add Product workflow with server-side validation (required fields, non-negative price/stock, unique SKU, valid category reference) and image upload support (PNG/JPG/JPEG/WEBP, safely renamed and stored server-side).
- Graceful frontend resilience — the UI falls back to placeholder categories and images if the backend is briefly unreachable, rather than breaking outright.

Code Submission (GitHub Link)

<https://github.com/Hrtss-lab/upskillcampus/blob/main/backend/app.py>

(Full project, including the frontend, is available at: <https://github.com/Hrtss-lab/upskillcampus>)

Report Submission (GitHub Link)

https://github.com/Hrtss-lab/upskillcampus/blob/main/AutoPartsHub_Mumo_USC_UCT.pdf

Proposed Design / Model

High Level Diagram

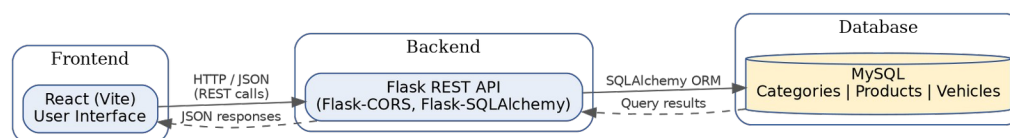


Figure 1: High-level architecture — React frontend, Flask REST API backend, and MySQL database.

Low Level Diagram

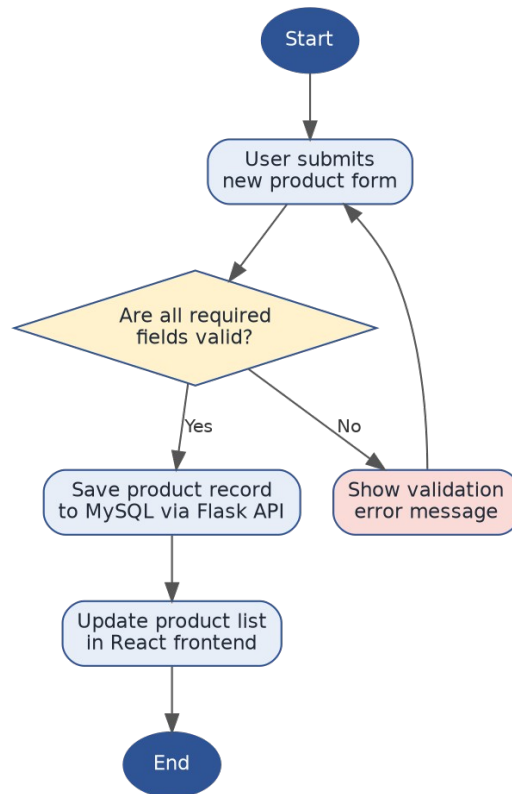


Figure 2: Add New Product flow — form submission, validation, save to database, and frontend list update.

Interfaces

The React frontend and Flask backend communicate over HTTP using JSON (and multipart/form-data for image uploads) through a REST API. The following endpoints are implemented:

Method	Endpoint	Purpose
GET	/api/health	Checks API and database connectivity
GET	/api/categories	Lists all categories, alphabetically
GET	/api/products	Lists all products, newest first
GET	/api/products/<id>	Retrieves a single product
POST	/api/products	Creates a product (multipart/form-data, optional image)
PUT	/api/products/<id>	Updates an existing product (JSON body)
DELETE	/api/products/<id>	Deletes a product
GET	/api/products/search?q=	Searches by name, brand, SKU or description
GET	/api/vehicles	Lists all vehicles, ordered by make/model/year

On the frontend, product creation is validated client-side before submission (required name/category/SKU, non-negative price and stock) as a first line of defence, with the same rules re-validated server-side as the authoritative check. Server responses use standard HTTP status codes — 400 for validation errors, 404 for missing resources, 409 for SKU conflicts, and 201/200 for successful creation/updates.

[Placeholder — the Vehicles endpoint currently returns data but is not yet linked to specific products in the frontend UI; this association is planned for a later phase (see Future Work Scope).]

Performance Test

Formal load and performance benchmarking has not yet been carried out, since the priority so far has been building correct, validated functionality. The main constraints identified for AutoParts Hub are data integrity (preventing invalid or duplicate records), input validation robustness, and — as the catalogue grows — query performance and search responsiveness. The design accounts for the first two today; the third is planned for a later phase once a realistic volume of product data exists to test against.

Test Plan / Test Cases

- Create a product with all required fields — expect success (201) and the new product appearing in the catalogue.
- Create a product missing a required field (name, SKU, category, or price) — expect a 400 error with a clear message.
- Create a product with a duplicate SKU — expect a 409 conflict, no duplicate record created.
- Create a product with a negative price or stock quantity — expect a 400 validation error.
- Upload an unsupported image format — expect a 400 error; upload a supported format — expect the image stored and served correctly via /uploads.
- Search for a product by partial name, brand, SKU and description — expect matching results in each case.
- Simulate the backend being unreachable — expect the frontend to show a connection error with a retry option rather than crashing.

Test Procedure

Testing so far has been manual: exercising the Add Product form and API endpoints directly through the running frontend, and verifying behaviour against the test cases above. The /api/health endpoint is used to confirm database connectivity independently of the product endpoints.

Performance Outcome

- Validation rules (required fields, non-negative price/stock, unique SKU, valid category reference) behave as expected on both the client and server.
- Image upload correctly rejects unsupported formats and safely renames accepted files to avoid collisions or path issues.
- Search correctly matches across name, brand, SKU and description fields.
- Frontend resilience was confirmed: when the backend is unavailable, the UI displays a clear error state with a retry action instead of failing silently.

[Placeholder — formal response-time/load testing (e.g. with a larger seeded dataset, or a tool such as Apache JMeter or a simple scripted load test) is planned once the core feature set is complete.]

My Learnings

Across the weeks completed so far, this internship has provided both conceptual grounding and hands-on, practical experience:

- Planning a full-stack application before implementing its complete functionality — defining objectives, entities and architecture up front, then building toward them incrementally.
- Designing and implementing a relational schema (Categories, Products, Vehicles) with SQLAlchemy, including foreign keys, uniqueness constraints and sensible defaults.

- Building a Flask REST API with proper separation of concerns — serialization helpers, layered validation, and meaningful HTTP status codes (400/404/409/201) rather than generic error responses.
- Handling file uploads securely — restricting extensions, using `secure_filename` and UUID-based renaming to avoid collisions or path-traversal issues, and serving uploaded images back through a dedicated static route.
- Managing non-trivial state in React with hooks (`useState`, `useEffect`, `useMemo`) — handling asynchronous data loading, derived/computed dashboard metrics, form state, and image preview via object URLs.
- Designing for resilience on the frontend — falling back to default categories/images and showing a clear retry option if the backend is briefly unreachable, instead of letting the UI break silently.
- Reinforcing core Java and object-oriented programming concepts (inheritance, polymorphism, abstraction, encapsulation) that underpin backend and Android development more broadly, and building a working understanding of full-stack architecture and HTML fundamentals through parallel coursework.

[To be expanded with career-relevant reflections once later weeks of implementation are complete.]

Future Work Scope

- Build out the Inventory, Sales, Invoices and Vehicles pages — navigation and layout are already wired up, but the underlying functionality is not yet implemented.
- Link Vehicles to Products in the frontend, so parts can be filtered or associated by compatible make/model/year (the Vehicle API already exists but isn't yet consumed by the UI).
- Split `App.jsx` into separate component files (currently a single large file) and consider introducing a routing library instead of state-based navigation, for maintainability as the app grows.
- Add authentication/authorisation if required for multi-user access.
- Add automated tests (backend unit tests for validation logic; frontend component/integration tests) to complement the manual testing done so far.
- Run formal performance/load testing once a realistic volume of product data is available.